

CendovaPlan — Installations- & Datenschutz-

Kurzinfo für die Klinik-IT

Diese Seite richtet sich an den/die IT-Beauftragte(n) und erklärt, **was das Programm ist, wie Patientendaten geschützt sind und was für die Installation auf einem Klinik-PC nötig ist.**

TL;DR

CendovaPlan ist eine **reine Browser-Anwendung ohne Server/Cloud**: Patienten- Röntgenbilder (DICOM) werden **nur lokal im Browser** verarbeitet und **verlassen den Rechner nie**. Für die Installation braucht es lediglich **Node.js + Git** und **ausgehenden HTTPS-Zugriff auf GitHub und npm** — ausschließlich, um den **Programmcode** zu laden/aktualisieren, **nie für Patientendaten**.

1 · Was es ist

- Browser-basiertes **Planungstool für Hüft-/Knie-Endoprothetik** auf DICOM-Röntgenbildern (Messungen, Implantat-Schablonen, PDF-Plan).
- Technisch eine **statische Client-Anwendung** (React/TypeScript, Vite). **Kein Backend, kein Server, keine Datenbank, keine Cloud.**
- **Wichtig**: Lern-/Eigenprojekt, **nicht CE-zertifiziert** → dient der **Planung/Erprobung**, ist kein Medizinprodukt und ersetzt keine Diagnose-/Behandlungsentscheidung.

2 · Datenschutz

- **Patientendaten bleiben lokal**: DICOM-Dateien werden im Browser des Klinik-PCs geöffnet und dort verarbeitet. Es findet **keine Übertragung, kein Upload, keine Speicherung außerhalb des Rechners** statt.
- **Nur localhost**: Die App läuft unter `http://localhost:5173` (127.0.0.1). Es wird **kein nach außen erreichbarer Port** geöffnet — von anderen Geräten/aus dem Netz ist nichts ansprechbar.
- **Die einzige Netzwerkverbindung** dient dem **Laden/Aktualisieren des Programms** (Quellcode + Programmbibliotheken) — siehe Abschnitt 4. **Über diese Verbindung werden niemals Patientendaten gesendet.**
- **Das Code-Repository enthält keine Patientendaten** — DICOMs sind per `.gitignore` generell ausgeschlossen.
- **Offline-fähig**: Nach der Erstinstallation kann das Programm ohne Internet betrieben werden; Internet ist nur für (optionale) Updates nötig.
- **Quelloffen/prüfbar**: Quellcode und Abhängigkeiten sind einsehbar und können von der IT auditert werden.

3 · Was am Klinik-PC gebraucht wird

Komponente	Zweck	Anmerkung
Node.js LTS (z. B. 20 oder 22)	Startet die lokale App	Standard-Installer; Admin-Recht nur einmalig zur Installation
Git	Holt/aktualisiert den Programmcode	Standard-Installer
Moderner Browser mit WebGL	Zeigt die App + GPU-Bildrendering	Microsoft Edge ist vorinstalliert und genügt

Ausgehender Netzwerkzugriff (nur HTTPS, nur für Updates — keine PHI):

- `github.com` — Programmcode. **Lesender** Zugriff auf das öffentliche Repository (`github.com/cendova/cendova-plan`) — kein Konto, kein Token, kein Schreibrecht vom Klinik-PC nötig.
- `registry.npmjs.org` — Programmbibliotheken (npm).

Nicht nötig: eingehende Ports, Server-/Dienst-Installation, Datenbank, Cloud-Konto, dauerhafte Admin-Rechte.

4 · Einrichtung

Empfohlen — Ein-Klick per USB-Installer: Den Ordner `installer\` vom USB-Stick öffnen und **Installieren.cmd** doppelklicken (am besten gemeinsam mit der IT). Das Skript erledigt alles:

- prüft/installiert **Node.js + Git** (via `winget`, mit IT-/UAC-Freigabe),
- holt CendovaPlan nach `%USERPROFILE%\CendovaPlan` (bzw. **aktualisiert** eine vorhandene Installation),
- installiert die Programmbibliotheken (`npm install`),
- legt die **Desktop-Verknüpfung** „CendovaPlan“ an.

Beim ersten Start kann die **Windows-Firewall** für den lokalen Server nachfragen → für „privat/localhost“ erlauben. Der verfolgte Branch ist standardmäßig `main` (anpassbar: `Installieren.cmd -Branch <name>`).

Manuell (Alternative): Node.js + Git installieren → `git clone <REPO-URL>`
`%USERPROFILE%\CendovaPlan` → im Ordner `scripts\create-desktop-shortcut.cmd` doppelklicken.

5 · Täglicher Betrieb

Doppelklick auf die Desktop-Verknüpfung. Das Launcher-Skript (`scripts\start-local.cmd`) macht automatisch:

1. **neueste Version holen** — `git pull --ff-only`
2. **Bibliotheken installieren/aktualisieren** — `npm install`
3. **App lokal starten und Browser öffnen** — `npm run dev -- --open` → `http://localhost:5173`

Danach das Röntgenbild per **Drag-&Drop (Datei, Ordner oder ZIP)** laden — es bleibt vollständig lokal. Schließen: das Konsolenfenster schließen bzw. `Strg+C` .

6 · Warum dieses Setup

- **Parallele Weiterentwicklung ohne Datentransfer:** Verbesserungen werden zu Hause entwickelt und ins Repository veröffentlicht; der Klinik-PC zieht sie beim nächsten Start automatisch. So kann am Klinik-PC mit **echten, lokal verbleibenden** Bildern getestet werden, **ohne Patientendaten hin- und herzuschieben**.

7 · Alternative, falls Internet/GitHub am Klinik-PC unerwünscht ist

Statt „Live-Pull“ kann zu Hause ein **fertiger, offline lauffähiger Build** erzeugt werden:

```
npm run build      # erzeugt den Ordner dist/ (statische Dateien)
```

Dieser `dist/` -Ordner wird per USB/Netzlaufwerk auf den Klinik-PC gebracht und dort mit einem **lokalen Static-Server** geöffnet (kein Internet, kein npm, kein Git am Klinik-PC nötig). Trade-off: Updates erfolgen dann **manuell** statt automatisch.

Bei Rückfragen der IT: Es handelt sich um eine lokale, quelloffene Client-Anwendung ohne Patientendaten-Übertragung. Der Quellcode ist öffentlich einsehbar (github.com/cendova/cendova-plan).